

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	ATHIN SIVA.S	Reg. No.:	192524237
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION**Industry Problem Solving Task**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				

Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO4 AT2- Industry Problem Solving Task OPEN ✓ Completed

1/1 answered

Review →

View Rubric

Criteria	Exemplary	Proficient	Developing	Beginning
Understanding of Industry Problem 20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding: some requirements unclear	Poor understanding: misinterprets problem context
Application of Engineering Concepts 25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design 25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools 15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis 10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism 5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. Problem Statement

The Travelling Salesman Problem (TSP) asks us to find the minimum-cost tour that visits every city exactly once and finally returns to the starting city. In this problem, the TSP is solved using two different approaches: Dynamic Programming (DP) and a Greedy heuristic. Their tour costs, execution efficiency, optimality, and scalability are then compared.

2. Cost Matrix

The question image does not contain a specific numerical cost matrix, so the following 4-city matrix is used as an illustrative input for the solution. The cities are A, B, C and D.

City	A	B	C	D
A	0	28	17	26
B	28	0	11	18
C	17	11	0	10
D	26	18	10	0

The matrix is symmetric, meaning the travel cost from A to B is the same as from B to A. The diagonal values are zero because no travel cost is required to remain in the same city.

3. Method 1 – Dynamic Programming

Dynamic Programming solves TSP by breaking the problem into overlapping subproblems. For each subset of visited cities, the algorithm stores the minimum cost of reaching a particular city. This avoids recalculating the same partial tours.

Let $DP[S][j]$ represent the minimum cost of starting from city A, visiting every city in subset S, and ending at city j. The recurrence is:

$$DP[S][j] = \min(DP[S - \{j\}][i] + \text{cost}[i][j])$$

The base case is $DP[\{A\}][A] = 0$. After all cities are visited, the cost of returning to A is added to obtain the complete tour.

4. DP Calculation for the Example

Starting from A, the DP method examines all relevant subsets and computes the minimum cost for each possible ending city. The minimum complete tour obtained is:

$$A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$$

$$\text{Cost} = A-C (17) + C-B (11) + B-D (18) + D-A (26) = 72$$

Therefore, the optimal TSP tour has a total cost of 72.

5. Method 2 – Greedy Heuristic

The Greedy heuristic makes the locally cheapest choice at every step. Starting from A, the algorithm selects the nearest unvisited city and continues until all cities have been visited. Finally, it returns to A.

1. Start at A.
2. From A, the cheapest unvisited city is C with cost 17.
3. From C, the cheapest unvisited city is D with cost 10.
4. From D, the only remaining unvisited city is B with cost 18.
5. Return from B to A with cost 28.

Greedy tour: $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$

Greedy cost = $17 + 10 + 18 + 28 = 73$

6. Python Program

```
from itertools import combinations
```

```
def tsp_dp(cost):
    n = len(cost)
    dp = {}

    for i in range(1, n):
        dp[(1 << i, i)] = cost[0][i]

    for size in range(2, n):
        for subset in combinations(range(1, n), size):
            mask = sum(1 << i for i in subset)
            for j in subset:
                prev_mask = mask ^ (1 << j)
                best = float('inf')
                for i in subset:
                    if i != j:
                        best = min(best,
                                   dp[(prev_mask, i)] + cost[i][j])
                dp[(mask, j)] = best

    full_mask = (1 << n) - 2
    return min(dp[(full_mask, j)] + cost[j][0]
               for j in range(1, n))
```

```
def tsp_greedy(cost):
    n = len(cost)
    visited = [False] * n
    current = 0
    visited[current] = True
    total = 0

    for _ in range(n - 1):
        next_city = -1
        best_cost = float('inf')

        for j in range(n):
```

```

        if not visited[j] and cost[current][j] < best_cost:
            best_cost = cost[current][j]
            next_city = j

    total += best_cost
    visited[next_city] = True
    current = next_city

total += cost[current][0]
return total

cost = [
    [0, 28, 17, 26],
    [28, 0, 11, 18],
    [17, 11, 0, 10],
    [26, 18, 10, 0]
]

print("DP Minimum Tour Cost =", tsp_dp(cost))
print("Greedy Tour Cost =", tsp_greedy(cost))

```

7. Sample Output

DP Minimum Tour Cost = 72

Greedy Tour Cost = 73

8. Comparison of DP and Greedy

Feature	Dynamic Programming	Greedy Heuristic	Observation
Solution quality	Optimal	Not always optimal	DP guarantees minimum cost
Tour cost	72	73	DP is better for this example
Basic idea	Evaluate subproblems	Make cheapest local choice	Different decision strategies
Time complexity	$O(n^2 2^n)$	$O(n^2)$	Greedy is much faster
Space complexity	$O(n 2^n)$	$O(n)$	DP requires much more memory
Scalability	Poor for large n	Good for larger n	Greedy is more practical

9. Efficiency Analysis

The Dynamic Programming solution has exponential complexity. Its time complexity is $O(n^2 2^n)$ and its space complexity is $O(n 2^n)$. It is therefore suitable mainly for small or moderate numbers of cities when an exact answer is required.

The Greedy nearest-neighbor heuristic requires only $O(n^2)$ time with a simple implementation and $O(n)$ additional space. It can handle considerably larger instances, but it does not guarantee the globally optimal tour.

10. Scalability Issues

TSP is an NP-hard optimization problem. As the number of cities increases, the number of possible tours grows extremely rapidly. Dynamic Programming reduces the brute-force search but still has exponential

growth because of the 2^n term. For example, doubling the number of cities can cause a very large increase in the number of DP states.

Greedy methods scale much better because they make a decision using only the current city and its unvisited neighbors. However, an early local choice may force an expensive edge later, which explains why the greedy tour can be slightly or substantially worse than the optimum.

11. Trade-off Between Optimality and Computation Time

Dynamic Programming prioritizes optimality. It explores enough combinations to guarantee the minimum tour cost, but this guarantee requires significantly more computation and memory.

The Greedy heuristic prioritizes speed and simplicity. It produces a valid tour quickly and is useful when the number of cities is large or when an approximate answer is acceptable. The main disadvantage is the absence of an optimality guarantee.

For the selected example, DP gives a cost of 72 while Greedy gives 73. Thus, Greedy is only 1 cost unit worse here, while being computationally much cheaper. In other inputs, the gap can be much larger.

12. Critical Evaluation

DP is preferable when exact optimality is essential and the number of cities is manageable.

Greedy is preferable when speed, low memory usage, and scalability are more important.

A greedy solution is not proof of optimality because local decisions do not always lead to a global optimum.

For practical large-scale TSP, more advanced approximation or metaheuristic techniques can be considered.

The choice of algorithm should depend on the required solution quality, available computation time, and problem size.

13. Conclusion

The Travelling Salesman Problem demonstrates the important DAA trade-off between exactness and efficiency. Dynamic Programming guarantees an optimal solution but has exponential time and space requirements. The Greedy heuristic is considerably faster and more scalable, but it may produce a non-optimal tour. For the given illustrative 4-city matrix, DP obtains the optimal cost of 72, whereas the Greedy approach obtains 73. Therefore, DP is the better choice for guaranteed optimality on small instances, while Greedy is a practical choice for larger instances where fast approximate solutions are acceptable.